

# 操作系统研讨课

## Course: B0911011Y

卢天越

2026 春季学期



# Project 2 A Simple Kernel

2026.3.27



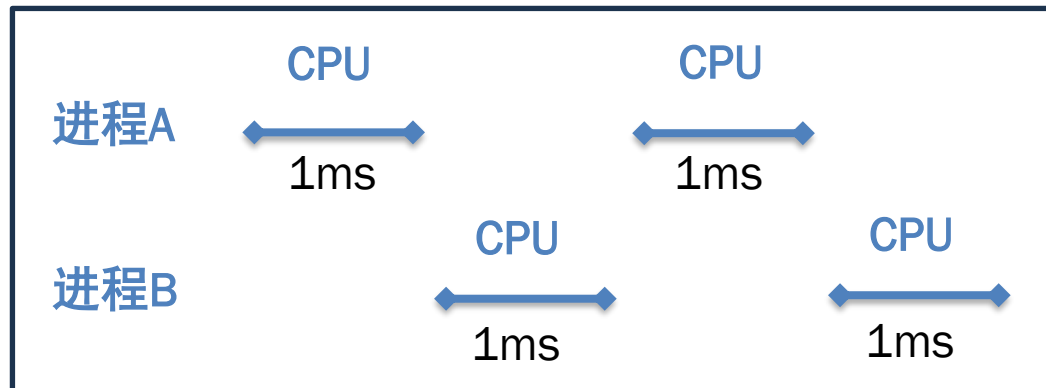
# 今日课程内容

- Project 2内容介绍
- Project 1 due
- 介绍开发板用法



# 操作系统功能：调度

- 完成P1之后，我们的操作系统已经进入内核
- 用户程序可以加载启动，但是一次只能**跑完一整个程序**
- 需要更强的操作系统功能：进程调度
  - 通过OS功能，让用户程序分时复用处理器资源



# P2任务1：非抢占式调度

- 如果希望强行打断用户进程的执行，需要硬件的中断：*抢占式调度*
- 我们先简化：非抢占式调度
  - 假如进程有绅士风度，**主动让出运行权**，操作系统需要做什么？
- **上下文切换 (context switch)**
  - 用户不知道其他进程的运行情况
- 代码框架中，包含了用户程序调用yield函数放弃运行权
  - 同学们需要实现yield函数功能，并完成上下文切换

```
for (int i = 0;; i++)  
{  
    sys_move_cursor(0, print_location);  
    printf("> [TASK] This task is to test schedule  
    sys_yield();  
}
```



# 上下文切换

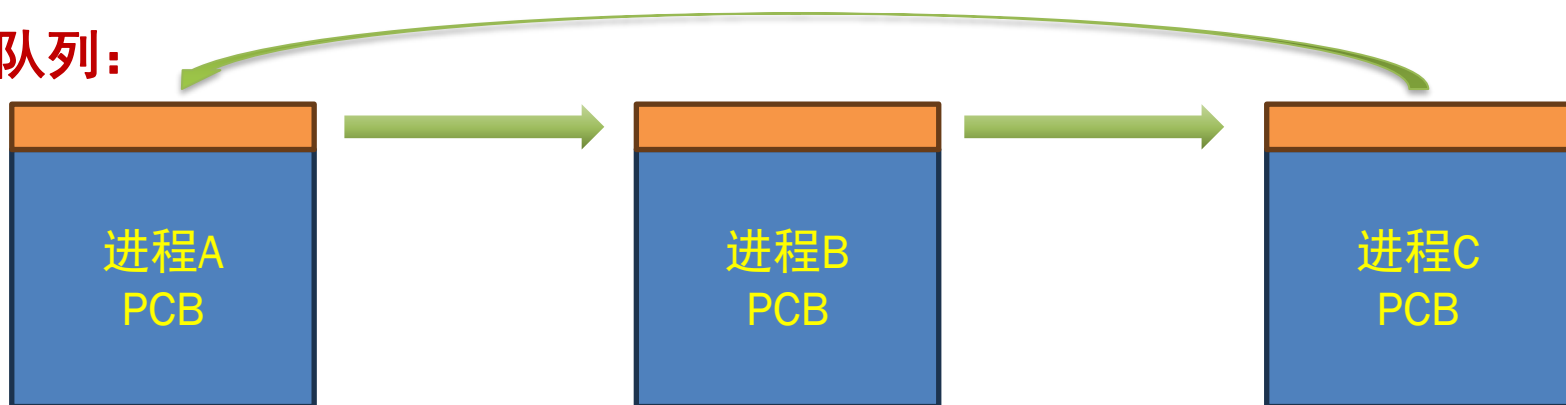
- 哪些上下文需要切换：寄存器
  - 全部的通用寄存器和部分控制状态寄存器
- 切换到哪里：内存
  - 保存上下文：寄存器->内存
  - 恢复上下文：内存->寄存器
- 内存的哪里：内核栈 (kernel stack)
  - 为什么是内核栈：符合先进先出的思想
  - 内核栈怎么分配：选定内存区域，每个进程4KB
    - 内存区域不和已有的代码冲突即可，进程的内核栈之间保证不冲突即可



# 调度算法与数据结构

- 调度 (schedule) : 决定下一个运行的程序是哪一个
  - 推荐使用简单的轮转算法, 公平调度
- 如何配合的使用数据结构?
  - 链表: 调度队列
  - 进程控制块 (Process Control Block, PCB)

调度队列:



# 进程控制块 (PCB)

- PCB结构体：包含哪些必要的成员变量？
  - PID
  - 内核栈地址
  - 链表所需的指针
- 后面会随功能扩展
  - *include/os/sched.h*

```
typedef struct pcb
{
    /* register context */
    // NOTE: this order must be preserved, which
    reg_t kernel_sp;
    reg_t user_sp;

    /* previous, next pointer */
    list_node_t list;

    /* process id */
    pid_t pid;

    /* BLOCK | READY | RUNNING */
    task_status_t status;

    /* cursor position */
    int cursor_x;
    int cursor_y;

    /* time(seconds) to wake up sleeping PCB */
    uint64_t wakeup_time;
} pcb_t;
```



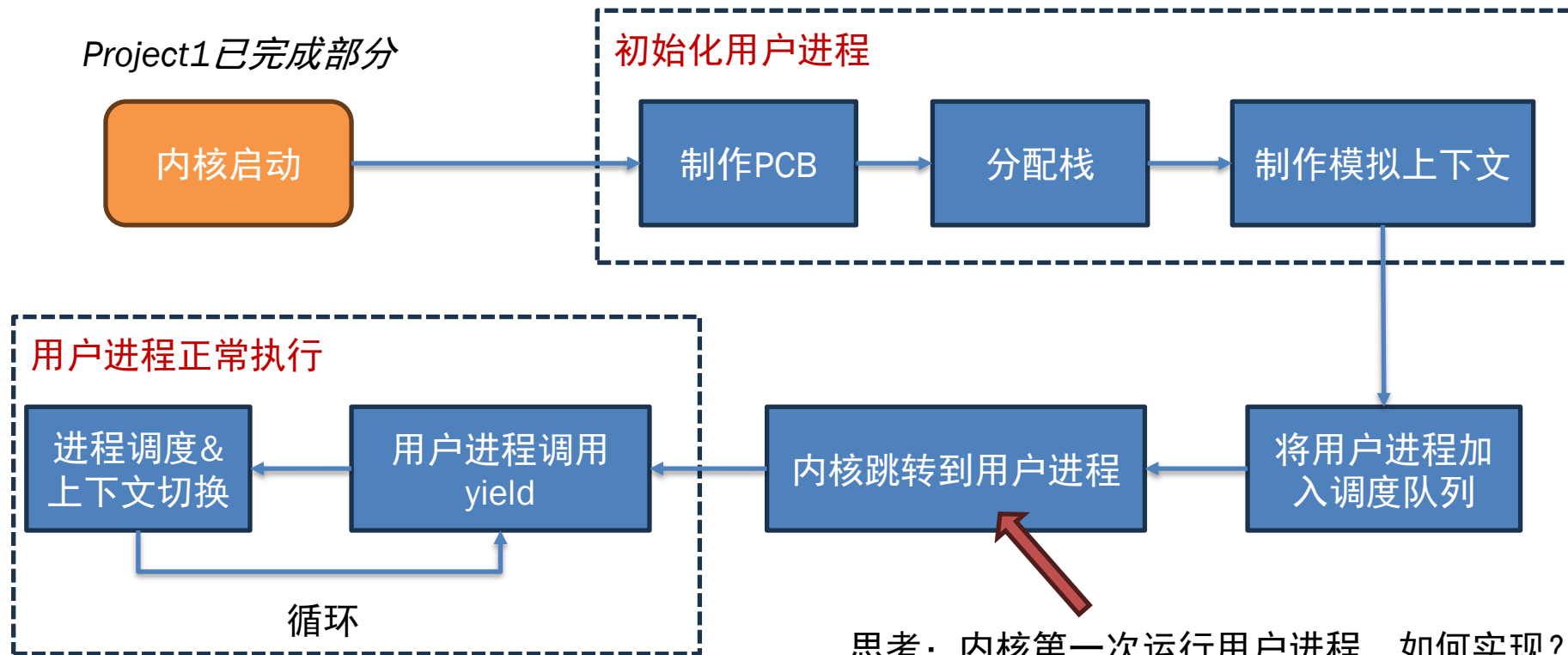


# 如果要切换的进程还没有运行过？

- 每次切换下一个进程的时候，都需要恢复该进程的上下文
- 但如果该进程从来没有运行过，没有上下文，如何处理？
  - 1、在PCB里增加标志位，特殊处理
  - 2、在内核初始化用户进程的时候，**制造模拟的上下文**
- 哪些寄存器是模拟上下文所需要的？
  - ra寄存器：用户程序的入口地址
  - sp寄存器：用户程序运行所需的栈
  - 将这些寄存器的值放在内核栈上存放上下文的位置，供调度时恢复上下文之后写入对应寄存器



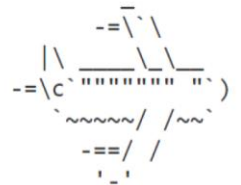
# 非抢占式调度小结



# 测试程序运行效果

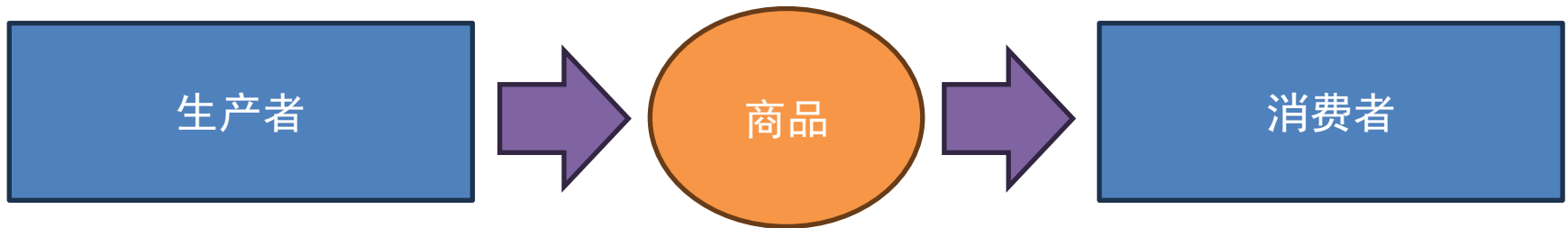
- 三个测试进程
  - print1、print2、fly
- 看到右边这样的效果则Project2的任务一完成
  - 两个print进程进度一致
  - 飞机在匀速飞行
  - 说明**调度公平**

```
> [TASK] This task is to test scheduler. (226)
> [TASK] This task is to test scheduler. (225)
```



# 任务2：互斥锁（mutex lock）

- 在进程运行过程中，可能出现多个进程需要对某个共享资源进行竞争的情况，需要保护共享资源的互斥访问
- 简单例子：生产者&消费者



– 生产者对商品+1，消费者对商品-1，同时访问数据可能会出错



# 互斥：申请锁失败则阻塞

- 如果一个进程申请锁，但是失败：
  - 自旋锁：不断重试，直到获取锁成功
    - 浪费CPU资源
  - 最好能释放运行权，交给其他能运行的程序来运行
- 互斥锁：一旦进程请求锁失败，那么该进程会自动被挂起到该锁的阻塞队列中，不会被调度器进行调度
  - 也是一种主动放弃运行权的情况
  - 与yield放弃运行权的区别：直到占用该锁的进程释放之后，该进程才回到调度队列中



# 互斥锁的用户接口

- mutex\_acquire
  - mutex\_acquire 成功则拥有锁
  - 失败则进入阻塞队列
- mutex\_release
  - 将阻塞队列的进程放出来

临界区代码

```
sys_move_cursor(0, print_location);
printf("> [TASK] Applying for a lock.\n");

sys_yield();

sys_mutex_acquire(mutex_id); ←

for (int i = 0; i < 5; i++)
{
    sys_move_cursor(0, print_location);
    printf("> [TASK] Has acquired lock and running. (%d)\n", i);
    sys_yield();
}

sys_move_cursor(0, print_location);
printf("%s", blank);

sys_move_cursor(0, print_location);
printf("> [TASK] Has acquired lock and exited.\n");

sys_mutex_release(mutex_id); ←

sys_yield();
```



# PCB加入锁功能

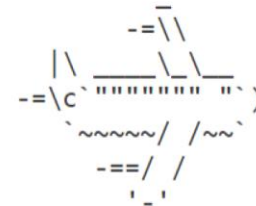
- 进程状态
  - 就绪态、阻塞态
- 进程拥有的锁
  - 记录该进程当前拥有的锁
  - 后面用于进程退出后释放锁
- *思考：进程能否同时拥有多把锁？*



# 测试程序运行效果

- 5个进程
  - 在任务一的基础上
  - 增加lock1和lock2
- 都运行起来则任务二完成
  - 不出现两个进程同时拥有锁
  - 任务一的三个进程现象不变

```
> [TASK] This task is to test scheduler. (96)
> [TASK] This task is to test scheduler. (96)
> [TASK] Has acquired lock and running.(2)
> [TASK] Applying for a lock.
```





# 任务3：系统调用

- P1的时候使用call bios\_func的方式调用一些操作系统服务
  - 但是用户程序不能随意跳转到内核地址
- 主动触发例外：通过硬件的例外机制触发强制跳转，由操作系统处理例外并返回，达到调用系统调用的效果
- 例外触发：ecall指令
  - 注意嵌入汇编写法



# 例外处理

- 例外发生后，硬件自动跳转到例外入口地址
  - svtec寄存器所指向的地址
- 多级处理：
  - 第一级：跳转到例外入口（硬件）
  - 第二级：通过scause寄存器判断例外类型
  - 第三级：以系统调用为例，根据系统调用号进行对应处理
- 例外返回：例外发生的地址记录在sepc寄存器
  - 使用sret指令，跳转回这个地址并返回用户态
  - 对于系统调用，返回之前sepc的值需要+4



# 任务3S：打印例外信息

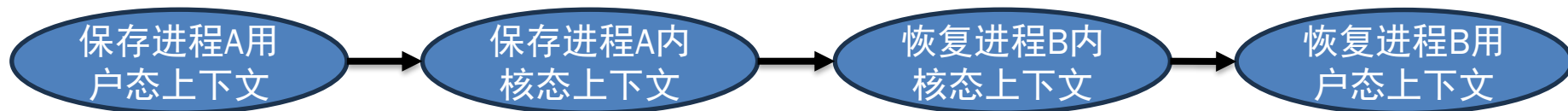
- S-core只要求在进入例外之后打印例外信息
  - 包括scause、sepc、stval这些CSR寄存器
- 做其他core的同学也可以在调试过程中加入这些打印信息
  - 确定例外是否被正确触发
  - 判断最后一次例外是由哪个地址触发的

CSR寄存器使用csrr和csrw指令来读写！



# 例外处理

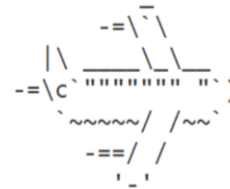
- 例外之后由于会触发强制跳转，所以需要保存用户态现场
  - 在调用sret指令返回之前恢复用户态现场
- 一个容易搞混的事情：
  - 进出内核态需要保存用户态上下文
  - 进程切换需要保存内核态上下文
  - 做一次sys\_yield系统调用会发生：



# 测试程序运行效果

- 实现多种系统调用
- 新增两个sleep测试
- 共7个进程参加调度
  - 注意本课程要求A-core任务用-O2编译
  - QEMU和板子的运行速度可能有明显差异

```
> [TASK] This task is to test scheduler. (92)
> [TASK] This task is to test scheduler. (90)
> [TASK] Has acquired lock and running.(4)
> [TASK] Applying for a lock.
> [TASK] This task is to test sleep. (2)
> [TASK] This is a thread to timing! (9/9142859 seconds).
```



# 任务4：抢占式调度

- 使用sys\_yield进行进程切换需要所有进程都有绅士风度
- 抢占式调度：由内核在定时器中断的时候进行进程切换
- 定时器中断处理：
  - 与系统调用很像的多级处理流程
    - 通过scause寄存器确定例外类型
    - 保存恢复上下文



# 定时器中断使能

- 当前的系统为什么不会在定时器到达一定时间之后就发生中断
- 使能/屏蔽中断：sie寄存器、sstatus寄存器
  - 当 *sstatus.SIE* 为 0 时，所有的中断都被屏蔽。当硬件发生中断时，硬件会自动将 *sstatus* 寄存器里面的 *SIE* 置为 0，将 *SPIE* 置为原来的 *SIE* 值。当执行 *sret* 时，硬件会将 *SPIE* 置为 1，*sstatus* 寄存器中的 *SIE* 置为原来的 *SPIE* 值。*sie* 寄存器不会变化。
  - 为什么设计成这么复杂？



# 定时器中断使能

- 屏蔽中断：定时器依然在走，到达特定时间依然会有中断信号产生，只是这个信号不处理
- 如果在使能中断的时候，这个中断信号早已到来，那么中断会立刻发生，因此：
  - 1、需要确保sret之前不会产生中断，由sret指令来真正使能中断。
  - 2、每次中断处理之后清中断
    - 注意清中断和屏蔽中断的概念区别，对于定时器中断来说，set\_timer设置下次中断时间即清中断



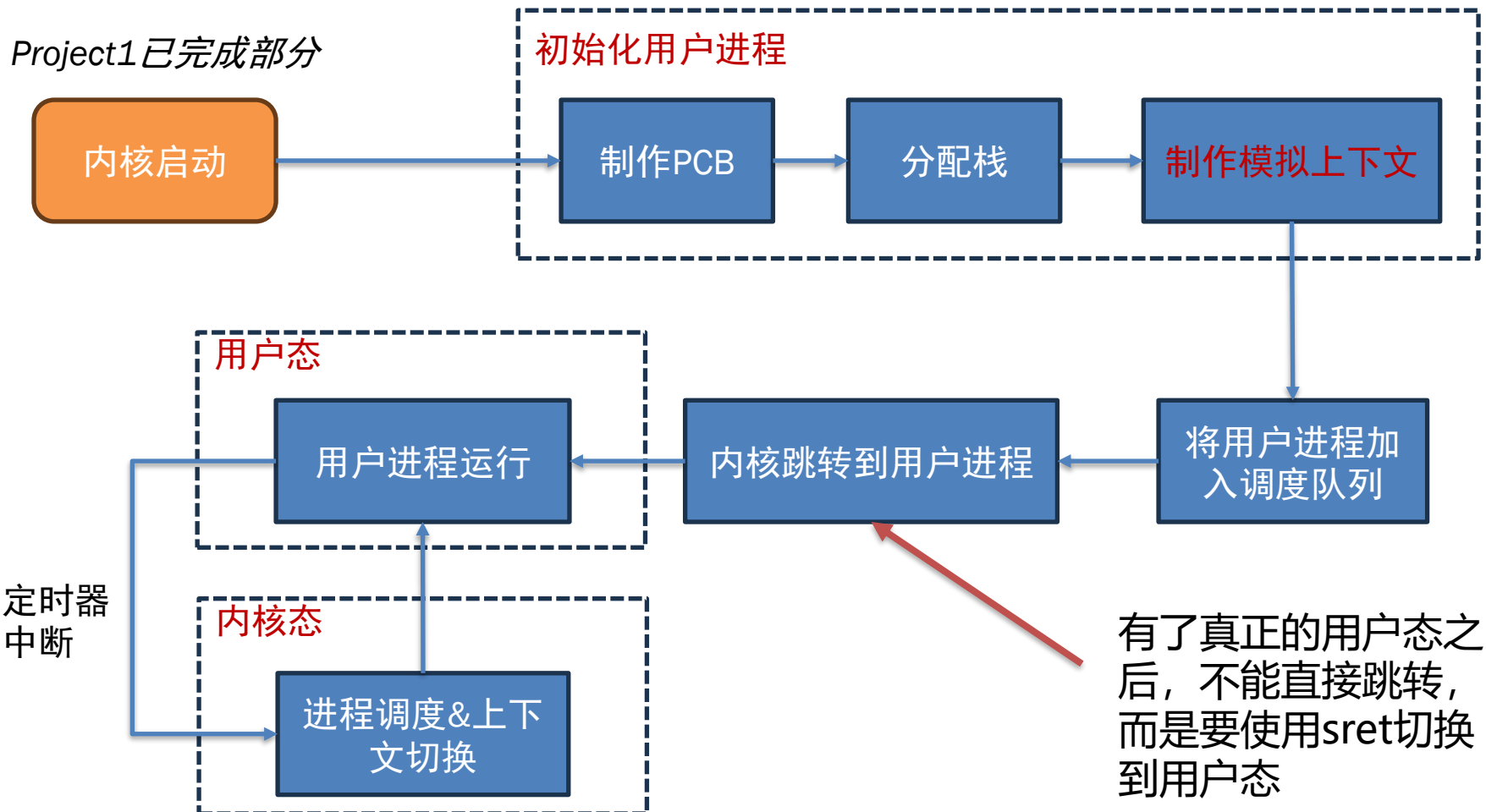


# 抢占式调度

- 在每次定时器中断的时候进行进程调度，就达到了抢占式调度的效果
  - 此时用户程序不再需要自己使用`sys_yield`放弃运行权了
  - 但是由于可能被中断的位置变多了，你的代码bug可能也会暴露



# 抢占式调度小结



# 任务5：复杂调度算法

- 截止任务4：所有任务公平调度，一次占用一个时间片
- 实现复杂调度：
  - 测试程序fly1~fly5，每个fly程序的飞机飞行速度和起始位置都不一样
  - 通过调度算法，使得所有飞机的飞行速度依然相同
    - 根据初始位置、速度和剩余距离调整时间片数量
    - 可以把时间片大小适当调小以实现精确控制



# 总结P2任务

- S-core任务：任务1、2、3S
  - 实现非抢占式调度和互斥锁，在例外处理时只打印例外错误信息
- A-core任务：任务1、2、3、4
  - 实现抢占式调度、互斥锁、系统调用功能
- C-core任务：任务1、2、3、4、5
  - 在A-core的基础上完成复杂调度功能



# 其他注意事项

- P2开始验收直接上板进行，注意QEMU的运行效果可能和板子不一样，也有一定概率出现QEMU运行通过板子不通过的情况
  - 由于时钟频率不同，有些bug可能在QEMU上不复现，可以通过改时间片大小的方式尝试复现
- **验收时会改几个时间片大小来验证**



# P2 Design review

- 介绍你的设计，覆盖以下问题：
  - 结合要实现非抢占式或抢占式调度回答：
    - 上下文切换的流程是什么样的
    - 初始化PCB和栈需要做什么事情
    - 上下文都保存哪些内容
  - 介绍你的进程调度算法和实现的数据结构
  - 如何实现一个进程被阻塞
  - 描述系统调用的处理流程
  - 初始化中断处理需要设置哪些寄存器，具体的值是什么



# 谢谢!

下节课: 4月3日, P2 design review  
Any questions?

